

知能情報実験 III（データマイニング班）
競馬複勝予測

グループの学籍番号列挙 245725D, 245304F, 245731J, 235751J

提出日：2026 年 6 月 9 日

目次

1	本実験向け補足	3
1.1	競馬複勝予測とは	3
2	実験計画と改善 (PDCA)	3
2.1	第 1 回: ベースラインモデルの構築 (Plan・Do)	3
2.2	第 2 回: 人気・単勝オッズの追加 (Do・Check)	4
2.3	第 3 回: 特徴量の見直し (Plan・Do)	4
2.4	第 4 回: 過去成績の追加 (Do)	4
2.5	第 5 回: 過去成績の相対化 (Do・Check)	4
2.6	第 6 回: 回収率による評価 (Do)	5
2.7	第 7 回: ハイパーパラメータ調整 (Act)	5
2.8	実験目的	5
2.9	実験手順	5
2.10	データセット	7
2.11	モデル	7
2.12	パラメータ調整	8
3	使用した特徴量	10
4	実験結果	12
4.1	実験 1: ベースライン	12
4.2	実験 2: 市場予想 (単勝オッズ・人気) の追加	13
4.3	実験 3: 市場予想を除外した特徴量	13
4.4	実験 4: 過去成績特徴量の追加	13
4.5	実験 5: 同一レース内での相対化	14
4.6	実験 6: 回収率による評価	14
4.7	参考: 単勝オッズ・人気を特徴量に含めたモデル	16
4.8	実験 7: パラメータ調整	17
4.9	scale_pos_weight	21
4.10	L2 正則化 (reg_lambda)	22
4.11	L1 正則化 (reg_alpha)	23
5	最終的な設定まとめ	25
6	考察	25

7	意図していた実行計画との違い	27
8	まとめ	28

1 本実験向け補足

1.1 競馬複勝予測とは

本グループでは、競馬における馬の複勝率予測を対象問題として設定した。複勝とは、出走した馬が3着以内に入るかどうかを予測する馬券の種類であり、単勝や馬連と比較して的中しやすい特徴を持つ。本テーマでは、過去の競馬データを用いて、各馬が複勝するか否かを機械学習によって分類することを目的とする。競馬では、馬の年齢や体重、レース距離、天候、馬場状態など多くの要因がレース結果に影響すると考えられている。しかし、人間がこれらの膨大な情報を総合的に判断して予測を行うことは容易ではない。そこで、機械学習を活用することで、多数の説明変数から複勝に関係する特徴を学習し、客観的な予測を行うことが可能となる。

本テーマに取り組む意義は、実際の競馬予想への応用だけでなく、多数の特徴量を持つ実データに対して分類モデルを適用し、その性能や特徴量の重要度を分析できる点にある。これにより、機械学習を用いた予測手法の有効性を検証するとともに、データ分析の実践的な知識を深めることができる。

2 実験計画と改善 (PDCA)

本研究では、一度モデルを作成して終わりではなく、実験結果を分析し、改善を繰り返すPDCAサイクルを意識して開発を進めた。各回の実験内容と、それを踏まえた改善内容を以下に示す。

2.1 第1回：ベースラインモデルの構築 (Plan・Do)

まず、競馬データ [1] をダウンロードし、最低限の特徴量として「馬体重」「天候」「馬齢」「距離」「芝・ダート区分」「着順」を抽出した。また、着順が3着以内であれば複勝フラグを1、それ以外を0として教師データを作成した。

その後、ロジスティック回帰、Random Forest[2]、XGBoost[3] の3種類のモデルを実装し、Accuracy、F1-score、ROC-AUC、Precision、Recall の5つの指標で性能評価を行った。さらに、特徴量重要度を確認し、予測に大きく影響する特徴量を分析した。

- **Check**：基本的な学習・評価環境を構築できたが、特徴量が少なく予測性能には改善の余地があることが分かった。
- **Action**：予測性能向上のため、新たな特徴量を追加することにした。

2.2 第2回：人気・単勝オッズの追加（Do・Check）

人気・単勝オッズは他の馬券購入者の予想が集約された情報であり、着順との相関が強く、最も有効な特徴量になると事前に予想されたため、これらを追加して再度学習・評価を行った。

- **Check**：モデル性能は向上したが、人気や単勝オッズは人の予想が反映された情報であり、本来予測したい馬自身の能力とは異なる情報に強く依存していることが分かった。また、これらの特徴量の説明力が極めて強いいため、予測結果がほぼ人気・オッズのみで決まってしまう、他の特徴量の寄与度やモデル間の性能差を適切に評価することが困難であった。
- **Action**：人の判断が介在する特徴量を除外し、馬自身の能力を表す特徴量のみで予測できるモデルを目指すことにした。

2.3 第3回：特徴量の見直し（Plan・Do）

人気と単勝オッズを削除し、生データに含まれる特徴量のみを用いてモデルを再構築した。また、生データに存在する他の特徴量も追加し、情報量の増加を図った。

- **Check**：人の予想情報に依存しないモデルとなったが、単一レースの情報だけでは馬の実力を十分に表現できないことが分かった。
- **Action**：各馬の過去成績を特徴量として追加することにした。

2.4 第4回：過去成績の追加（Do）

馬ごとの過去成績を特徴量として追加した。具体的には、過去の平均着順や複勝率などを計算し、現在のレース予測に利用した。

- **Check**：過去成績を追加することで、馬の実力をより適切に表現できるようになった。しかし、絶対値だけではレース内での相対的な強さを十分に表現できていないことが分かった。
- **Action**：同一レース内で比較しやすい特徴量へ変換することにした。

2.5 第5回：過去成績の相対化（Do・Check）

過去成績を同一レース内で比較できるように加工した。具体的には、過去成績をレースごとに標準化するとともに、順位付けを行い、0から1の範囲に正規化した特徴量を作成した。

- **Check**：各馬の実力を同一レース内で比較しやすくなり、特徴量の表現力が向上した。
- **Action**：モデル性能だけでなく、実際の馬券購入を想定した評価指標も導入することにした。

2.6 第6回：回収率による評価（Do）

Accuracy や ROC-AUC などの機械学習指標に加え、各モデルについて馬券の回収率を算出し、実運用を意識した評価を行った。

- **Check**：分類性能が高いモデルと、実際の回収率が高いモデルは必ずしも一致しないことが分かった。
- **Action**：回収率も考慮しながら、モデル性能をさらに改善するため、ハイパーパラメータ調整を実施することにした。

2.7 第7回：ハイパーパラメータ調整（Act）

各モデルについてハイパーパラメータを調整し、デフォルト設定との性能比較を行った。

- **Check**：パラメータ調整による性能向上の効果を確認し、各モデルの特性や最適な設定について理解を深めることができた。
- **Action**：今後は特徴量のさらなる改善や特徴量選択、アンサンブル手法なども検討し、回収率を含めた総合的な性能向上を目指す。

2.8 実験目的

本実験の目的は、競馬の過去データを用いて機械学習モデルを構築し、出走馬が複勝（3 着以内に入着）するかどうかを予測できるかを検証することである。競馬のレース結果には、馬齢、馬体重、距離、天候、馬場状態など多くの要因が関係していると考えられるため、これらの特徴量から複勝の可否を予測する分類モデルを作成する。また、複数の特徴量を用いた機械学習によってどの程度の予測性能が得られるのかを評価するとともに、どの特徴量が複勝予測に大きく影響しているのかを分析することも目的とする。さらに、異なる機械学習手法や前処理方法を比較し、それらが予測精度に与える影響を検証する。これらの実験を通して、競馬における複勝予測の実現可能性を確認するとともに、実データを対象とした機械学習モデル構築の有効性について考察する。

2.9 実験手順

第三者が実験を再現するための手順を以下に示す。

1. GitHub リポジトリを取得する。

```
git clone <リポジトリ URL>
cd horse_racing
```

2. 必要なライブラリをインストールする。

```
uv sync
```

3. Kaggle から以下のファイルをダウンロードし、data ディレクトリに配置する。

- 19860105-20210731_race_result.csv
- 19860105-20210731_odds.csv

4. 特徴量作成 (最初に 1 回だけ実行)

```
./run_scripts/build_features.sh
```

5. 各実験は run_scripts/ 以下のシェルスクリプトにより、3 モデル (XGBoost, Random Forest, ロジスティック回帰) をまとめて実行できる。

実験 1

```
./run_scripts/ex1.sh
```

実験 2

```
./run_scripts/ex2.sh
```

実験 3

```
./run_scripts/ex3.sh
```

実験 4

```
./run_scripts/ex4.sh
```

実験 5

```
./run_scripts/ex5.sh
```

実験 6

ex6_1.sh は実験 5 と同じ特徴量で回収率を計算する。

ex6_2.sh は参考実験として実験 5 + 単勝オッズ・人気で回収率を計算する。

```
./run_scripts/ex6_1.sh
```

```
./run_scripts/ex6_2.sh
```

実験 7

ハイパーパラメータ探索を実行する。XGBoost のパラメータ調整を自動化するシェルスクリプトである。実行すると、指定したパラメータ (例えば木の深さ) について候補値ごとに学習・評価を繰り返し、結果の一覧表 (CSV) とグラフ (PNG) を results/sweep/以下に出力する。

出力されたグラフを見て最良の値を決定し、その値を固定したうえで次のパラメータのスイープを実行する。これを木の木数、木の深さ、`learning_rate`、`scale_pos_weight`、L2 正則化、L1 正則化の順に繰り返し、パラメータを段階的に決定した。

```
./run_scripts/ex7.sh
```

2.10 データセット

https://www.kaggle.com/datasets/takamotoki/jra-horse-racing-dataset?select=19860105-20210731_race_result.csv

本実験では、以下の 2 つのファイルを利用する。

1. 19860105-20210731_race_result.csv

各レースに出走した馬ごとの情報が記録されているファイルである。馬体重、馬体重増減、馬齢、競馬場、距離、天候、馬場状態、騎手など、多数の説明変数が含まれている。

2. 19860105-20210731_odds.csv

各レースのオッズ情報が記録されているファイルである。本研究では単勝オッズなどの情報を取得するために利用した。

2.11 モデル

- ロジスティック回帰
- XGBoost
- RandomForest

本実験では、ロジスティック回帰、Random Forest、XGBoost の 3 種類のモデルを比較した。

ロジスティック回帰は線形モデルであり、解釈のしやすさから他の複雑なモデルとの性能比較におけるベースラインとして採用した。

Random Forest は決定木を用いた bagging 系のアンサンブル手法であり、過学習に強く、特徴量重要度による解釈も容易であることから、非線形な関係や特徴量間の相互作用を捉える手法として採用した。

XGBoost は決定木を用いた boosting 系のアンサンブル手法であり、各木が前の木の誤差を逐次的に補正していく仕組みを持つため、Random Forest とは異なる学習方式による性能傾向の違いを比較する目的で採用した。同様の勾配ブースティング木の実装として LightGBM も候補に挙がったが、LightGBM の強みである大規模データでの高速学習は、本実験で扱うデータ規模（数十万件程度）では恩恵が限定的であると考えられた。また、本実験では第 7 回実験のようにハイパーパラメータを 1 項目ずつ段階的に調整する座標降下的な手順を採る計画であったため、パラメータの種類や相互作用についてチーム内で事前に把握しやすく、調整結果の妥当性を検討しやすい

XGBoost を採用することとした。

2.12 パラメータ調整

2.12.1 方針

時間的制約から、パラメータ調整は XGBoost のみを対象とした。調整は 1 項目ずつ値を振り、分類指標 (Accuracy, F1, ROC-AUC, Precision, Recall) と複勝回収率 (モデル 1 位・人気 1 位・モデル上位 3 頭・人気上位 3 頭) の変化をグラフで確認しながら、最善と判断した値を次の項目のベースラインに固定して進める、座標降下的な手順を取った。項目の順序は、木の本数 → 木の深さ → learning_rate → scale_pos_weight → L2 正則化 → L1 正則化である。

評価にあたっては、回収率の絶対的な最大値だけでなく、AUC・F1・Recall とのトレードオフを重視した。回収率がほぼ変わらないにもかかわらず分類指標 (特に Recall) が大きく悪化する場合は、過学習のサインとみなし、より保守的な値を採用する方針とした。具体的な結果・考察・採用値は次の「実験結果」章にまとめる。

図 1 に、座標降下的に進めた各フェーズの候補値と採用値の流れを示す。各フェーズの採用値は次フェーズのベースライン条件として固定され、順に木の本数 → 木の深さ → learning_rate → scale_pos_weight → L2 正則化 → L1 正則化の順で調整を進めた。

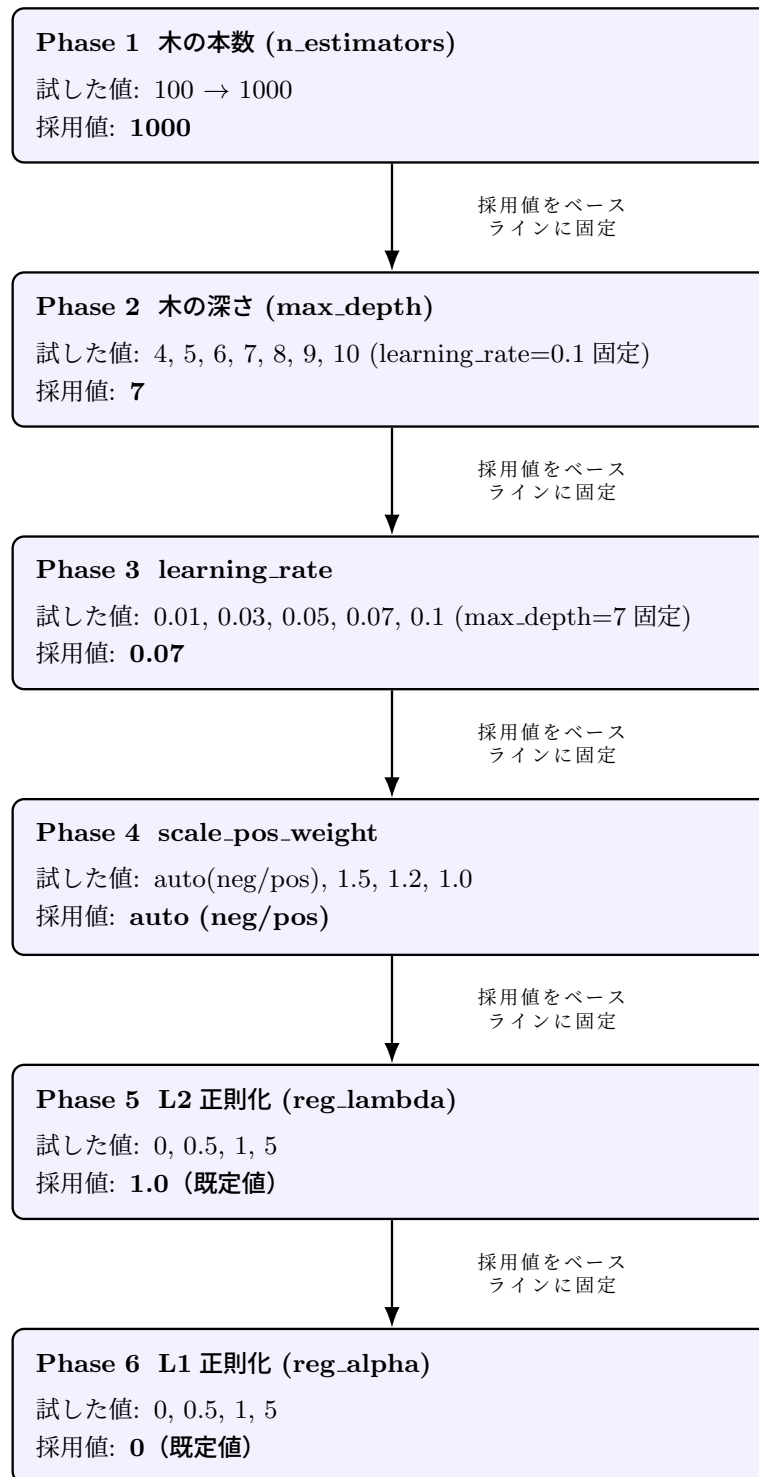


図1 パラメータ調整の手順 (座標降下的アプローチ)。各フェーズのパラメータ名・試した値・採用値を示す。

2.12.2 木の本数 (n_estimators)

100 → 1000 に変更した。木の本数を増やすこと自体に悪影響はないと判断し、個別の検証・グラフ化は行わなかった。

2.12.3 木の深さ (max_depth)

候補として 4~10 を設定し、learning_rate はデフォルト値 (0.1) に固定して実施した。

2.12.4 learning_rate

候補は 0.1 以下 (0.01, 0.03, 0.05, 0.07, 0.1) とした。初回は max_depth の調整結果を反映する前だったため、暫定的に depth=10 で実施した。しかし depth と learning_rate には交互作用があり、片方を固定したまま他方だけを見て結論を出すのは適切でないと判断し、max_depth=7 を確定させたうえで再度スweepを行った。

2.12.5 scale_pos_weight

候補として auto(neg/pos, 既定)、1.5、1.2、1.0 を設定し、max_depth=7、learning_rate=0.07 で実施した。

2.12.6 L2 正則化 (reg_lambda)

候補として 0, 0.5, 1, 5 を設定し、max_depth=7、learning_rate=0.07、scale_pos_weight=auto で実施した。

2.12.7 L1 正則化 (reg_alpha)

候補として 0, 0.5, 1, 5 を設定し、max_depth=7、learning_rate=0.07、scale_pos_weight=auto で実施した。

3 使用した特徴量

本研究で使用した特徴量は、数値型についてはそのまま入力し、カテゴリ型についてはカテゴリ数に応じて変換を行った。具体的には、カテゴリ数が 20 以下の特徴量は One-Hot エンコーディングによりカテゴリごとの二値変数 (0/1) に変換し、20 を超える特徴量は Ordinal Encoding により整数値に変換した。本研究で使用したカテゴリ変数はいずれもカテゴリ数が 20 以下であったため、すべて One-Hot エンコーディングを適用している。

使用した特徴量を表 1(基本特徴量)、表 2(レース条件に関する特徴量)、表 3(過去成績に基づく特徴量)、表 4(同一レース内での相対化特徴量) に示す。

表 1 基本特徴量

特徴量名	説明	モデルへの入力形式
馬齢	競走馬の年齢	数値 (そのまま)
馬体重	レース当日の馬体重	数値 (そのまま)
馬体重増減	前走からの馬体重の増減	数値 (そのまま)
斤量	負担重量	数値 (そのまま)
性別	牡・牝・セン	One-Hot(3 カテゴリ)
距離 (m)	レース距離	数値 (そのまま)
芝・ダート区分	芝コースかダートコースか	One-Hot(2 カテゴリ)
枠番	出走時の枠番号	数値 (そのまま)

表 2 レース条件に関する特徴量

特徴量名	説明	モデルへの入力形式
競馬場名	開催競馬場	One-Hot(10 カテゴリ)
馬場状態	良・稍重・重・不良	One-Hot(4 カテゴリ)
天候	レース当日の天候	One-Hot(5 カテゴリ)
右左回り・直線区分	コースの回り方向	One-Hot(3 カテゴリ)
距離帯	距離を 5 区分にカテゴリ化 (短距離・マイル・中距離・中長距離・長距離)	One-Hot(5 カテゴリ)
レースクラス	新馬・未勝利・1〜3 勝クラス・オープンの 6 区分	One-Hot(6 カテゴリ)
レースグレード	G1・G2・G3・L・なしの 5 区分	One-Hot(5 カテゴリ)
休養日数	前走からの日数	数値 (そのまま)

表 3 過去成績に基づく特徴量

特徴量名	説明	モデルへの入力形式
過去 3 走・5 走平均着順	直近の平均着順	数値 (そのまま)
過去 3 走・5 走複勝率	直近の複勝 (3 着以内) 率	数値 (そのまま、0～1)
過去 3 走・5 走平均上り	直近のラスト 3F 平均タイム	数値 (そのまま)
騎手複勝率	騎手の通算複勝率	数値 (そのまま、0～1)
調教師複勝率	調教師の通算複勝率	数値 (そのまま、0～1)
芝・ダート別複勝率	芝/ダート別の複勝率	数値 (そのまま、0～1)
距離帯別複勝率	距離帯別の複勝率	数値 (そのまま、0～1)
競馬場別複勝率	競馬場別の複勝率	数値 (そのまま、0～1)

表 3 のすべての過去成績に基づく特徴量は、対象レースより過去のデータのみを用いて算出しており、未来の情報が混入しないよう shift 処理を施している。

表 4 同一レース内での相対化特徴量

特徴量名	説明	モデルへの入力形式
順位率	同一レース内でのパーセンタイル順位 (0 に近いほど上位)	数値 (そのまま、0～1)
標準化	同一レース内での標準化得点 (Z スコア)	数値 (そのまま)

表 3 の過去成績特徴量および馬体重・斤量・馬齢・休養日数について、同一レースに出走する馬同士で比較した相対値を追加で作成した。

4 実験結果

4.1 実験 1: ベースライン

使用した特徴量: 馬齢, 馬体重, 距離 (m), 芝・ダート区分
結果を表 5 に示す。

表 5 実験 1 の結果

モデル	Accuracy	F1	ROC-AUC	Precision	Recall
XGBoost	0.4794	0.3544	0.5595	0.2427	0.6565
Random Forest	0.5012	0.3363	0.5408	0.2367	0.5804
ロジスティック回帰	0.4637	0.3544	0.5569	0.2401	0.6762

4.2 実験 2: 市場予想 (単勝オッズ・人気) の追加

実験 1 の特徴量に加え、単勝オッズ・人気を追加した。
結果を表 6 に示す。

表 6 実験 2 の結果

モデル	Accuracy	F1	ROC-AUC	Precision	Recall
XGBoost	0.7390	0.5456	0.8100	0.4393	0.7196
Random Forest	0.7551	0.4730	0.7611	0.4449	0.5048
ロジスティック回帰	0.7101	0.5368	0.8068	0.4115	0.7719

4.3 実験 3: 市場予想を除外した特徴量

単勝オッズ・人気を除外し、代わりに斤量, 馬体重増減, 枠番, 休養日数, 性別, 競馬場名, 馬場状態, 天候, 右左回り・直線区分, 距離帯, レースクラス, レースグレードを追加した。
結果を表 7 に示す。

表 7 実験 3 の結果

モデル	Accuracy	F1	ROC-AUC	Precision	Recall
XGBoost	0.5448	0.3724	0.6009	0.2660	0.6203
Random Forest	0.7145	0.2233	0.5655	0.2737	0.1885
ロジスティック回帰	0.4681	0.3660	0.5770	0.2471	0.7053

4.4 実験 4: 過去成績特徴量の追加

実験 3 の特徴量に加え、過去平均着順, 過去複勝率, 調教師複勝率, 騎手複勝率, 距離帯別複勝率, 過去平均上り, 芝・ダート別複勝率, 競馬場別複勝率を追加した。
結果を表 8 に示す。

表 8 実験 4 の結果

モデル	Accuracy	F1	ROC-AUC	Precision	Recall
XGBoost	0.7069	0.4806	0.7449	0.3913	0.6230
Random Forest	0.7712	0.4106	0.7356	0.4674	0.3662
ロジスティック回帰	0.7025	0.4667	0.7295	0.3827	0.5981

4.5 実験 5: 同一レース内での相対化

過去成績や馬体重などの特徴量について、同一レース内での順位率・標準化を追加した。結果を表 9 に示す。

表 9 実験 5 の結果

モデル	Accuracy	F1	ROC-AUC	Precision	Recall
XGBoost	0.7110	0.4957	0.7612	0.3997	0.6524
Random Forest	0.7761	0.4358	0.7525	0.4825	0.3973
ロジスティック回帰	0.7028	0.4864	0.7492	0.3898	0.6466

4.6 実験 6: 回収率による評価

実験 5 のモデルを用いて、閾値方式による複勝回収率を評価した。モデルの予測確率が一定の閾値を超えた場合にのみ馬券を購入したと仮定し、複勝回収率を評価した。購入方法として、以下の 2 種類を比較した。

- **全頭方式:** 閾値を超えた馬を、1 レースにつき複数頭でもすべて購入する方式 (該当馬が 0 頭のレースは購入しない)。
- **1 位方式:** 各レースで予測確率が最も高い馬のみを対象とし、その馬の予測確率が閾値を超えている場合にのみ購入する方式 (1 レースにつき購入は 0 頭または 1 頭のみ)。

いずれの方式も、1 頭 (1 点) あたり 100 円を購入したものと仮定し、以下の式で回収率を算出した。

$$\text{回収率} = \frac{\text{総払戻金}}{\text{総購入金額}} \times 100$$

なお、閾値を高く設定するほど購入対象となる頭数 (レース数) は減少するため、回収率とあわせて実際に賭けた頭数・購入レース数も併記する。

XGBoost

表 10 に全頭方式、表 11 に 1 位方式の結果を示す。

表 10 実験 6:XGBoost 回収率 (閾値&全頭方式)

閾値	0.5	0.6	0.7	0.8	0.9
回収率 (%)	80.05	80.43	81.17	82.84	85.65
賭けた頭数	475,808	317,233	177,845	68,606	8,332

表 11 実験 6:XGBoost 回収率 (閾値&1 位方式)

閾値	0.5	0.6	0.7	0.8	0.9
回収率 (%)	81.16	81.24	81.47	82.70	85.61
購入レース数	97,535	96,248	86,245	51,705	8,081

Random Forest

表 12 に全頭方式、表 13 に 1 位方式の結果を示す。

表 12 実験 6:Random Forest 回収率 (閾値&全頭方式)

閾値	0.5	0.6	0.7	0.8	0.9
回収率 (%)	80.09	80.91	82.66	83.83	89.44
賭けた頭数	240,000	109,079	37,422	7,935	557

表 13 実験 6:Random Forest 回収率 (閾値&1 位方式)

閾値	0.5	0.6	0.7	0.8	0.9
回収率 (%)	80.28	80.90	82.40	83.70	89.39
購入レース数	93,657	68,701	31,154	7,527	555

ロジスティック回帰

表 14 に全頭方式、表 15 に 1 位方式の結果を示す。

表 14 実験 6: ロジスティック回帰 回収率 (閾値&全頭方式)

閾値	0.5	0.6	0.7	0.8	0.9
回収率 (%)	79.33	79.31	80.09	81.19	84.39
賭けた頭数	483,452	299,497	160,847	62,786	8,634

表 15 実験 6: ロジスティック回帰 回収率 (閾値&1 位方式)

閾値	0.5	0.6	0.7	0.8	0.9
回収率 (%)	79.81	79.91	80.36	81.18	84.43
購入レース数	97,572	95,383	84,069	51,026	8,539

4.7 参考: 単勝オッズ・人気を特徴量に含めたモデル

実験 5 の特徴量に加えて単勝オッズおよび人気を追加したモデルについても、同様の方法で複勝回収率を算出した。なお、これらのモデルは人気 (市場予想) そのものを特徴量として利用しているため、モデルの予測が人気の情報を強く反映しやすい点に留意されたい。

4.7.1 XGBoost

表 16 に全頭方式、表 17 に 1 位方式の結果を示す。

表 16 参考:XGBoost(オッズ・人気を含む) 回収率 (閾値&全頭方式)

閾値	0.5	0.6	0.7	0.8	0.9
回収率 (%)	82.07	82.58	83.70	85.64	88.61
賭けた頭数	473,021	343,980	219,883	106,985	25,104

表 17 参考:XGBoost(オッズ・人気を含む) 回収率 (閾値&1 位方式)

閾値	0.5	0.6	0.7	0.8	0.9
回収率 (%)	84.15	84.15	84.21	85.31	88.66
購入レース数	97,595	97,571	96,330	77,999	24,759

4.7.2 Random Forest

表 18 に全頭方式、表 19 に 1 位方式の結果を示す。

表 18 参考:Random Forest(オッズ・人気を含む) 回収率 (閾値&全頭方式)

閾値	0.5	0.6	0.7	0.8	0.9
回収率 (%)	82.33	83.66	85.19	86.90	90.18
賭けた頭数	283,057	164,133	76,504	23,721	2,754

表 19 参考:Random Forest(オッズ・人気を含む) 回収率 (閾値&1 位方式)

閾値	0.5	0.6	0.7	0.8	0.9
回収率 (%)	83.63	83.87	85.01	86.89	90.10
購入レース数	97,403	90,379	60,840	22,644	2,752

4.7.3 ロジスティック回帰

表 20 に全頭方式、表 21 に 1 位方式の結果を示す。

表 20 参考: ロジスティック回帰 (オッズ・人気を含む) 回収率 (閾値&全頭方式)

閾値	0.5	0.6	0.7	0.8	0.9
回収率 (%)	80.63	81.17	82.16	84.06	91.43
賭けた頭数	528,115	393,795	244,212	74,233	91

表 21 参考: ロジスティック回帰 (オッズ・人気を含む) 回収率 (閾値&1 位方式)

閾値	0.5	0.6	0.7	0.8	0.9
回収率 (%)	83.09	83.09	83.09	84.03	91.43
購入レース数	97,595	97,595	97,559	66,058	91

4.8 実験 7: パラメータ調整

各パラメータのスweep結果を以下に示す。

n_estimators は検証を行っていないため結果は掲載しない。木の本数を増やすこと自体は、Random Forest のような bagging 系のアンサンブル手法と同様、XGBoost のような boosting 系の手法においても、他のハイパーパラメータ（木の深さ、learning rate 等）と比べて過学習への影響が相対的に小さいと考えられる。特に、learning rate（学習率）を小さく設定した状態で木の本数を増やすことで、1 本あたりの木の寄与を抑えながら段階的に学習を進めることができるため、

木の本数のみを増やしても過学習が急激に進行しにくいと考えられる。本実験でも、後続のフェーズで learning rate を 0.01~0.1 の範囲で調整しており、小さい learning rate との組み合わせによって木の本数を増やしたことによる過学習の影響は限定的であると判断した。以上の理由から、木の本数についてはデフォルトの 100 から 1000 へ増やす一方、個別の検証・グラフ化は省略し、後続のパラメータ（木の深さ以降）の調整に時間を割く方針とした。

なお、第 7 回実験で用いたモデルは、実験 3~5 とは異なり、特徴量に人気および単勝オッズを含めている。これは、実際に人気（市場予想）に従って馬券を購入した場合と比較して、モデルによる予測がどの程度改善しているかを確認するため、意図的に含めたものである。そのため、本章で示す回収率は、モデルが人気・単勝オッズという市場予想の情報も利用した上での結果であり、市場予想を除いた特徴量で評価した実験 3~5 とは前提が異なる点に留意されたい。

4.8.1 木の深さ (max_depth)

スワイプ結果を表 22 に示す。

表 22 max_depth スワイプ結果

値	Accuracy	F1	ROC-AUC	Precision	Recall	1 位回収率	上位 3 頭回収率
4	.7411	.5469	.8117	.4417	.7177	84.63	82.79
5	.7439	.5460	.8100	.4446	.7073	84.65	83.07
6	.7479	.5442	.8074	.4487	.6914	84.86	83.17
7	.7538	.5403	.8033	.4551	.6645	84.95	83.17
8	.7611	.5355	.7994	.4642	.6325	84.87	83.25
9	.7700	.5282	.7952	.4772	.5915	84.97	83.25
10	.7781	.5192	.7918	.4914	.5503	84.84	83.41

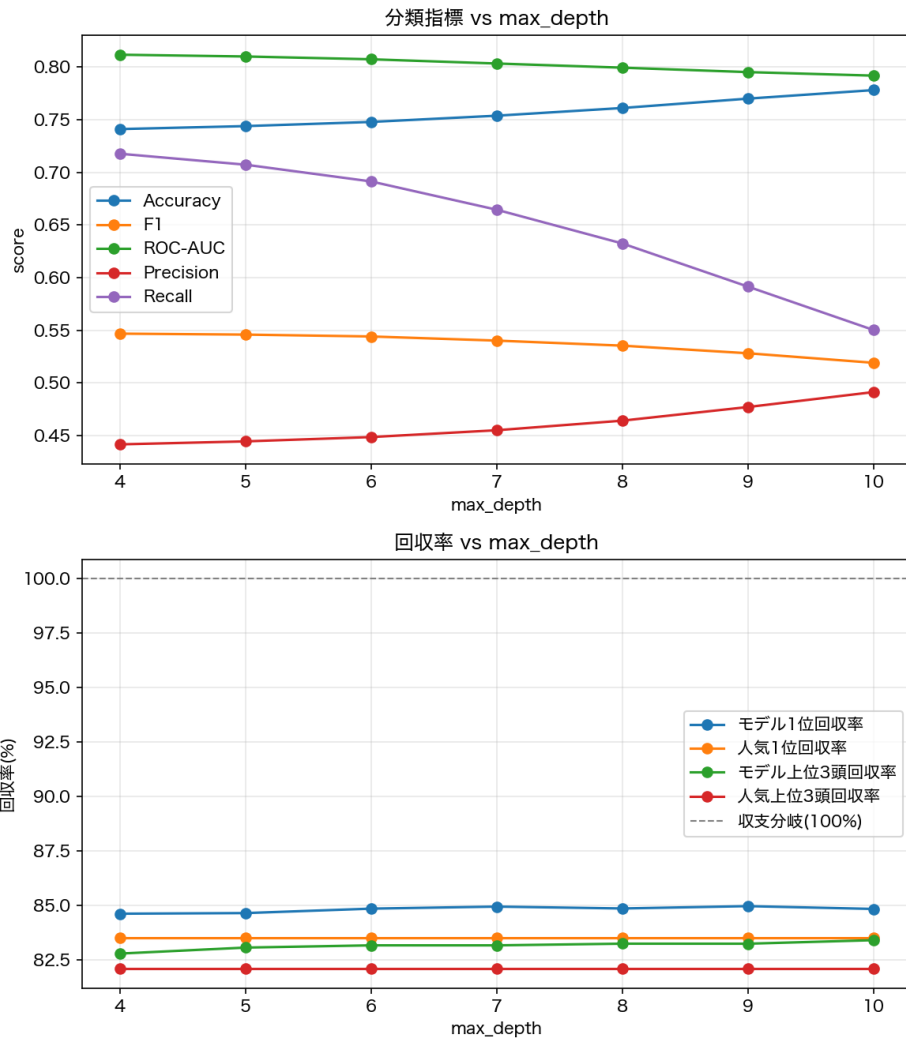


図2 分類指標・回収率 vs max_depth (learning_rate=0.1)

■採用値 max_depth = 7。回収率をほぼ落とさずに分類指標を保てる境界として採用した。

4.8.2 learning_rate

スニープ結果 (max_depth=7、採用)

スニープ結果を表 23 に示す。

表 23 learning_rate スイープ結果 (max_depth=7、採用)

値	Accuracy	F1	ROC-AUC	Precision	Recall	1 位回収率	上位 3 頭回収率
0.01	.7413	.5477	.8127	.4421	.7197	84.19	82.53
0.03	.7443	.5469	.8113	.4451	.7089	84.59	82.81
0.05	.7477	.5458	.8094	.4488	.6964	84.75	82.91
0.07	.7505	.5440	.8071	.4518	.6837	84.81	83.18
0.10	.7538	.5403	.8033	.4551	.6645	84.95	83.17

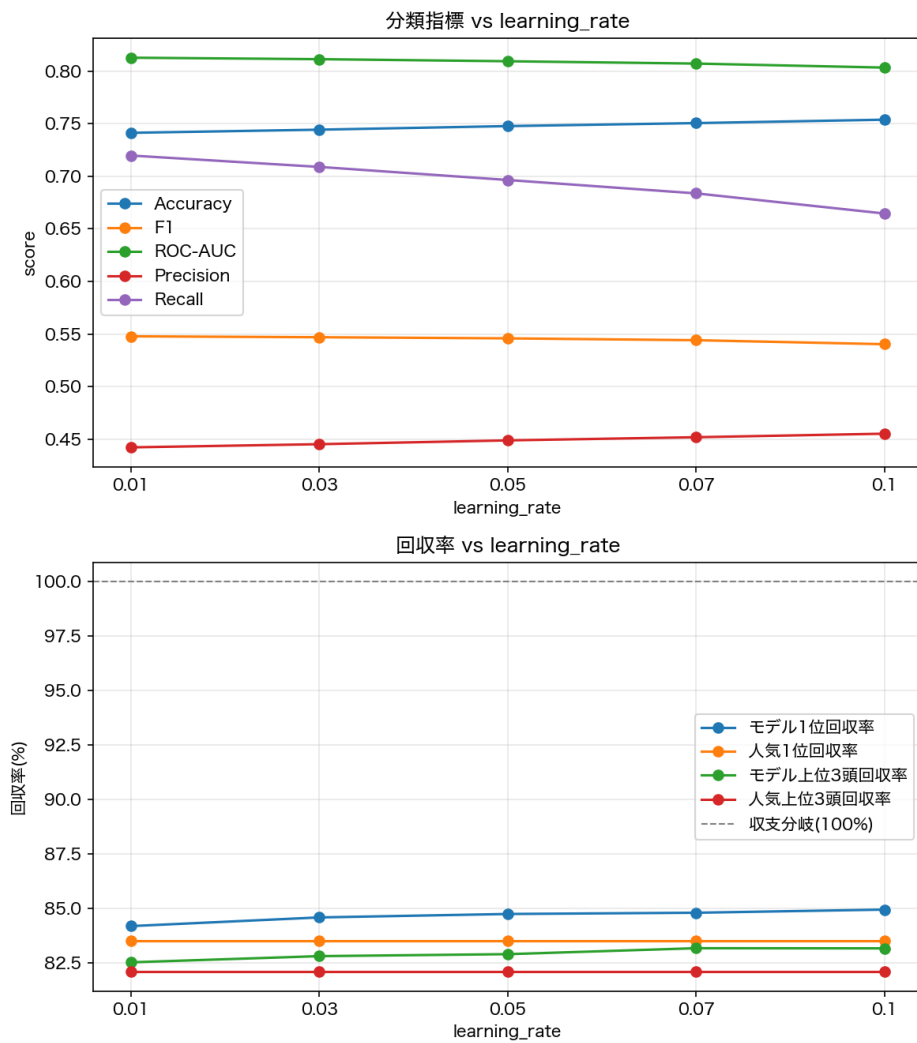


図 3 分類指標・回収率 vs learning_rate (max_depth=7、採用)

■採用値 learning_rate = 0.07。

4.9 scale_pos_weight

スイープ結果を表 24 に示す。

表 24 scale_pos_weight スイープ結果

値	Accuracy	F1	ROC-AUC	Precision	Recall	1 位回収率	上位 3 頭回収率
auto (neg/pos)	.7505	.5440	.8071	.4518	.6837	84.81	83.18
1.5	.8029	.5038	.8095	.5575	.4595	85.01	83.00
1.2	.8084	.4674	.8098	.5916	.3863	84.73	83.01
1.0	.8101	.4308	.8099	.6199	.3301	85.04	83.06

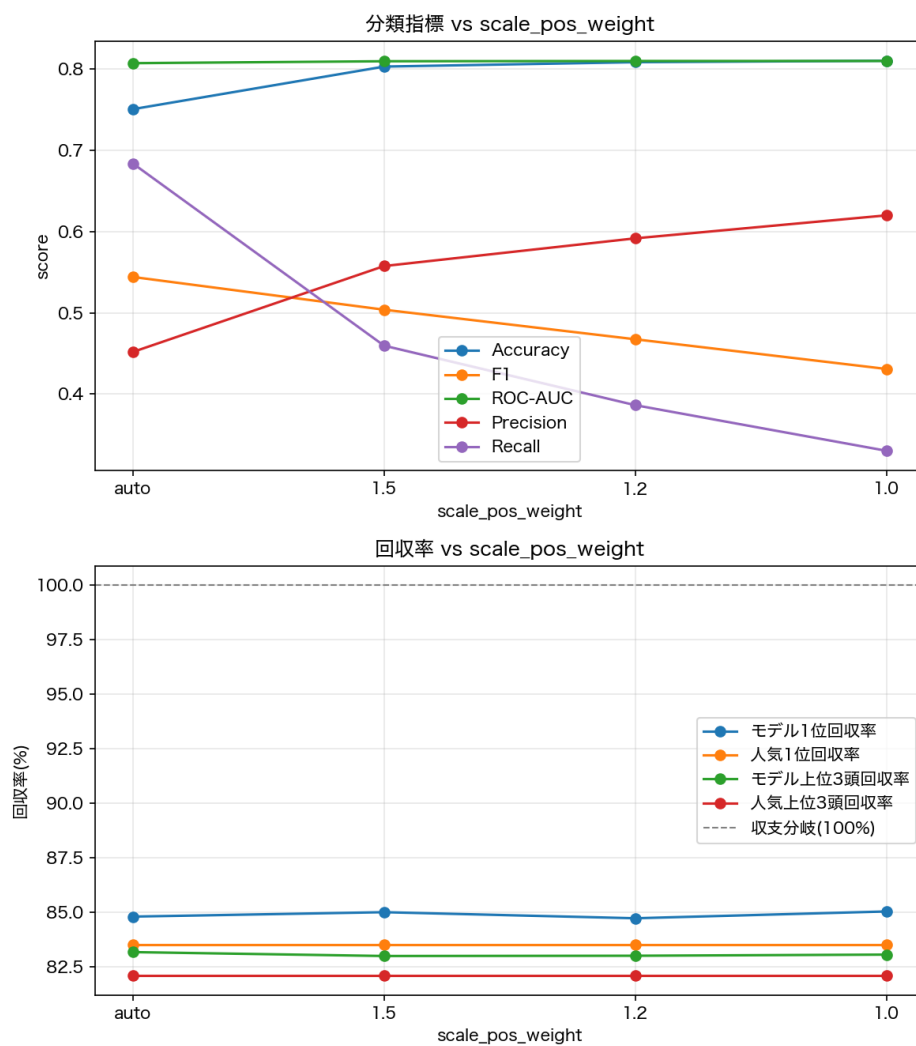


図 4 分類指標・回収率 vs scale_pos_weight (max_depth=7, learning_rate=0.07)

■採用値 `scale_pos_weight = auto (neg/pos)`。固定値は回収率をほぼ変えずに Recall を大きく犠牲にするため、既定の `auto` を維持した。

4.10 L2 正則化 (`reg_lambda`)

スイープ結果を表 25 に示す。

表 25 `reg_lambda` スイープ結果

値	Accuracy	F1	ROC-AUC	Precision	Recall	1 位回収率	上位 3 頭回収率
0.0	.7499	.5427	.8064	.4508	.6818	84.76	83.18
0.5	.7503	.5433	.8068	.4514	.6823	84.73	83.17
1.0	.7505	.5440	.8071	.4518	.6837	84.81	83.18
5.0	.7501	.5450	.8081	.4514	.6876	84.73	83.07

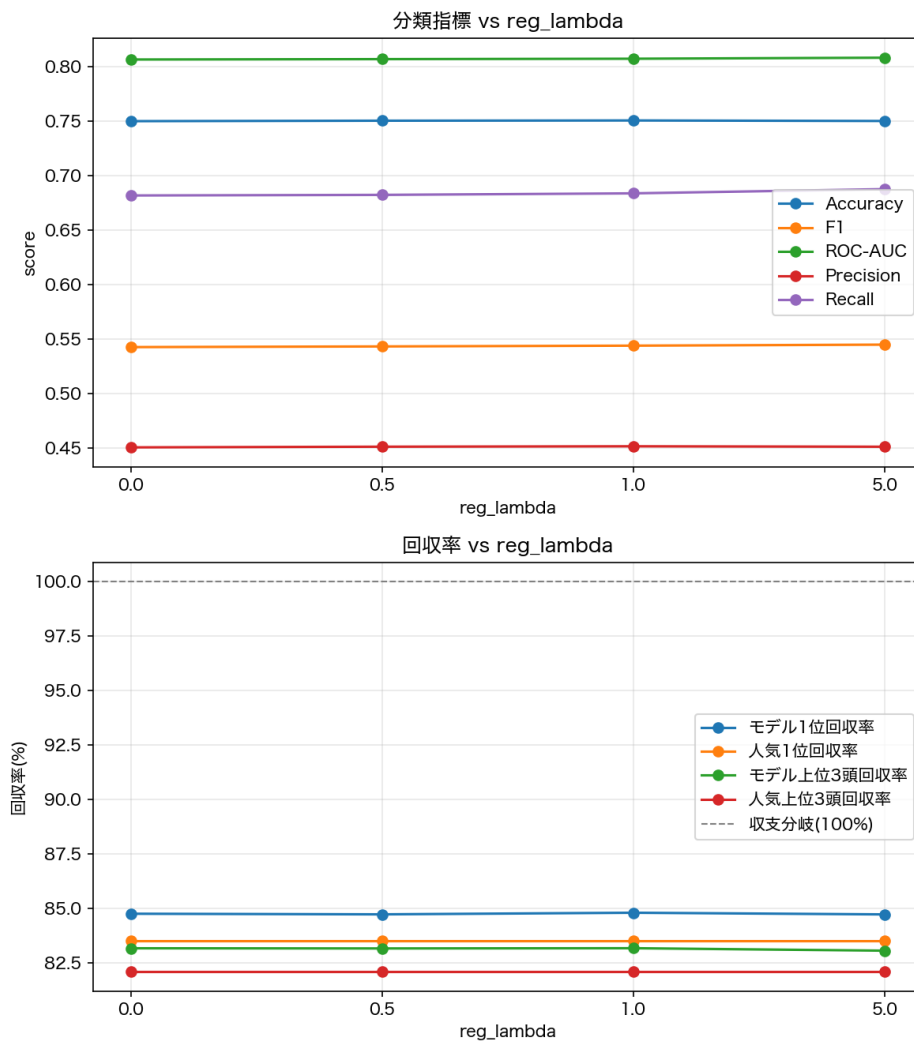


図5 分類指標・回収率 vs reg_lambda (max_depth=7, learning_rate=0.07, scale_pos_weight=auto)

■採用値 reg_lambda = 1.0 (既定値)。差が誤差レベルであり、変更する積極的な理由がないため既定値を維持した。

4.11 L1 正則化 (reg_alpha)

スweep結果を表 26 に示す。

表 26 reg_alpha スイープ結果

値	Accuracy	F1	ROC-AUC	Precision	Recall	1 位回収率	上位 3 頭回収率
0.0	.7505	.5440	.8071	.4518	.6837	84.81	83.18
0.5	.7506	.5437	.8070	.4518	.6824	84.85	83.09
1.0	.7509	.5439	.8071	.4522	.6824	84.76	83.13
5.0	.7512	.5448	.8081	.4527	.6841	85.09	83.22

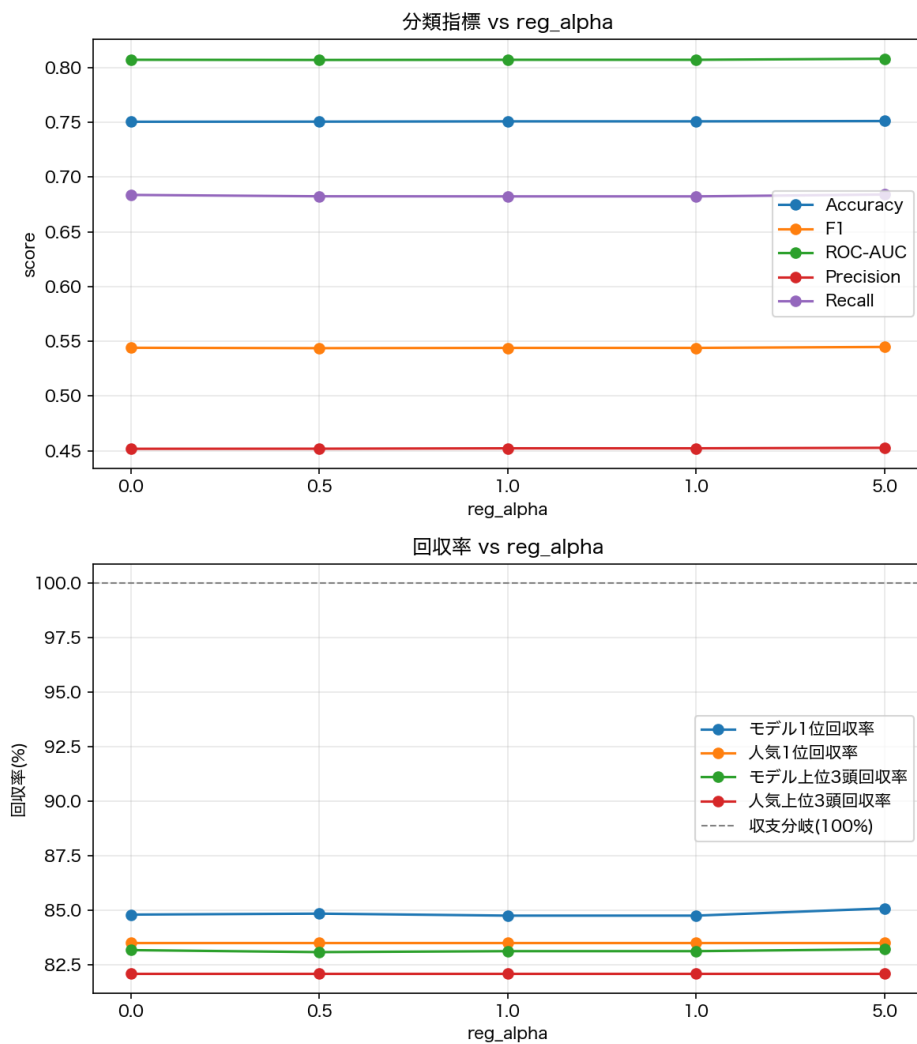


図 6 分類指標・回収率 vs reg_alpha (max_depth=7, learning_rate=0.07, scale_pos_weight=auto)

■採用値 reg_alpha = 0 (既定値)。reg_alpha=5.0 がわずかに全指標で上回るが、差はごくわずかでありモデルを複雑にする積極的な理由がないため、既定値を維持した。

5 最終的な設定まとめ

最終的に採用した XGBoost のハイパーパラメータと、その採用根拠を表 27 にまとめる。

表 27 最終的な XGBoost のハイパーパラメータ設定

パラメータ	採用値	根拠 (要約)
n_estimators	1000	デフォルトの 100 から増加。過学習リスクなしと判断し検証は省略。
max_depth	7	回収率は depth=6 以降ほぼ頭打ちなのに対し、AUC/F1/Recall は深くするほど悪化。回収率をほぼ落とさず分類指標を保てる境界として採用。
learning_rate	0.07	depth=10 で行った初回スweepでは rate=0.05 が回収率のピークだったが、depth=7 で再スweepすると傾向が変化。0.1 は 1 位回収率が最大だが AUC/F1/Recall を明確に犠牲にするため、回収率をほぼ落とさず分類指標を保てる 0.07 を採用。
scale_pos_weight	auto (neg/pos)	固定値 (1.5, 1.2, 1.0) は AUC にほぼ影響しないが F1・Recall を大きく悪化させ、上位 3 頭回収率も低下する。既定の auto(fold 毎に neg/pos を計算) を維持。
reg_lambda (L2)	1.0 (既定値)	0~5 の範囲で全指標がほぼ横ばい。効果は無視できる水準のため既定値を維持。
reg_alpha (L1)	0 (既定値)	reg_lambda と同様、0~5 の範囲でほぼ横ばい (reg_alpha=0 の基準値は reg_lambda スweep時と完全一致し条件の整合性を確認済み)。効果は無視できる水準のため既定値を維持。

6 考察

本研究では、ベースラインモデルの構築から特徴量の追加・改善、実運用を想定した回収率評価、ハイパーパラメータ調整に至るまで、一度の実験で終わらせず結果を分析しながら改善を繰り返す PDCA サイクルを実践した。分類指標の向上のみを目的とするのではなく、実データに内在する特徴量設計上の課題や、評価指標そのものの妥当性にまで踏み込んで検証できた点に、本テーマに取り組んだ意義があると思う。使用した特徴量は表 1~表 4 に示す通り基本情報・レース条件・

過去成績・相対化特徴量の4系統に整理されており、これらを段階的に組み合わせることで、どの情報が複勝予測に寄与するのかを切り分けて確認できた。

第1回実験では、馬齢・馬体重・距離・芝ダート区分といった最低限の特徴量のみでベースラインモデルを構築した(表5)。しかし、いずれのモデルも ROC-AUC が 0.56 前後にとどまり、少数の基本情報だけでは複勝の可否を十分に説明できないことが明らかになった。

第2回実験では、他の馬券購入者の予想が集約された単勝オッズ・人気を特徴量に追加したところ、表6に示す通り ROC-AUC が最大 0.81 まで大きく向上した。しかし、この結果は本来検証したかった「馬自身の能力に基づく予測」とは異なり、人間の予想情報にモデルが強く依存してしまうという問題を含んでいた。また、オッズ・人気の説明力があまりに強いため、他の特徴量の寄与度やモデル間の性能差を適切に評価することも困難であった。これは、目的を精査せずに予測性能の高さのみを追い求めた結果生じたつまづきであり、以降は「馬自身の能力を表す特徴量のみで予測する」という当初の問題設定に立ち返り、オッズ・人気を除外する方針へ修正した。

第3回実験では、オッズ・人気を除外し、斤量や馬体重増減、レース条件に関する特徴量を追加してモデルを再構築した。ところが表7に示す通り、Random Forest を中心に Recall や F1 が第2回実験から大きく低下し(Random Forest の Recall は 0.5048 から 0.1885 まで低下)、性能面では一時的に後退する結果となった。これは、単一レースの静的な情報だけでは馬の実力を十分に表現できていないことを示しており、失敗というよりも、次に取り組むべき課題、すなわち過去の傾向を表す特徴量の不足を明確にする契機となった。

第4回実験では、過去3走・5走の平均着順、複勝率、上りタイム、騎手複勝率、調教師複勝率などの過去情報を特徴量として追加した。その結果、表8に示す通り第3回実験と比較して各モデルの性能が向上し、特に XGBoost では ROC-AUC が 0.6009 から 0.7449 へ向上した。これは、競走馬の現在の状態だけでなく、過去の成績や得意条件などの傾向が複勝予測に有効であったためだと考えられる。

第5回実験では、既存の特徴量を同一レース内で相対化した特徴量(表4)を追加した。その結果、表9に示す通り XGBoost では ROC-AUC が 0.7449 から 0.7612 へ向上した。競馬では、馬体重や過去成績などの絶対値だけではなく、同じレースに出走する馬との相対的な能力差も結果に影響すると考えられる。そのため、レース内での位置づけを表す特徴量が有効に働いたと考えられる。

第6回実験では、予測確率に閾値を設定し、実際の馬券購入を想定した複勝回収率による評価を導入した。表11、表13、表15に示す通り、閾値を高く設定するほど回収率が向上する傾向が確認された。これは、予測確率が高い馬に限定することで、モデルが複勝と判断した馬の中でも、より確信度の高い馬を選択できたためだと考えられる。一方で、閾値を高くすると購入対象となる馬(レース)が減少するため、安定した収益を得るためには回収率だけでなく購入機会も考慮する必要がある。また、この評価を通じてモデルごとの性能傾向の違いも明らかになった。XGBoost は ROC-AUC が高く、予測確率の順位付け性能に優れていた一方、表12・表13に示す通り回収率評価では Random Forest が最も高い結果となった。このことから、分類性能が高いモデルが必ずしも収益性の高いモデルになるとは限らず、実際の購入戦略を考慮した評価が重要であることが分

かった。さらに、表 17～表 21 に示した参考実験 (オッズ・人気を特徴量に含めたモデル) では、いずれの閾値でも本来のモデルを上回る回収率 (1 位方式で最大 90% 超) が得られており、市場予想を排除して「馬自身の能力」のみで予測するという本研究の方針が、収益性の観点では一定の代償を伴っていることも確認された。

第 7 回実験では、XGBoost を対象にハイパーパラメータ調整を行った。図 2 に示す通り、木の深さ (max_depth) を増やすほど AUC・F1・Recall は単調に悪化する一方、回収率は depth=6 付近ではほぼ頭打ちとなり、6 から 10 まで増やしてもモデル 1 位回収率は +0.1pt 未滿、モデル上位 3 頭回収率も +0.2pt 程度しか伸びなかった。回収率がほとんど変わらないにもかかわらず判別力と Recall だけが大きく犠牲になることから、深さを増やしすぎる設定は過学習方向に寄りすぎていると判断し、max_depth=7 を採用した。learning_rate についても図 3 に示す通り同様の構図が見られ、max_depth=7 で再スweepした結果、rate=0.1 は 0.07 と比べてモデル 1 位回収率がわずかに (+0.15pt) 上回るのみで、AUC・F1・Recall (特に Recall は +1.9pt) はいずれも 0.07 が明確に上回ったため、回収率をほぼ落とさず分類指標を保てる rate=0.07 を採用した。scale_pos_weight については図 4 に示す通り AUC への感度は低い一方で F1・Recall は auto (既定) から離れるほど急激に悪化し (Recall は .684 から .330 へ半減以下)、回収率で見ても auto が大きく劣ることはなかったため、既定の auto を維持した。L1・L2 正則化 (reg_lambda, reg_alpha) は、図 5、図 6 に示す通りいずれも 0～5 の範囲で AUC・F1・回収率とも横ばいであり、正則化の効果はほぼ見られなかった。当初は大幅な性能向上を期待してパラメータ調整に取り組んだが、実際には既定値からの改善幅はごくわずかにとどまった。この結果は、想定していたほどの成果が得られなかったという意味では一つのつまずきであるが、同時に「今回使用したデータではモデルのパラメータ調整よりも、入力する特徴量の設計が性能に大きな影響を与える」という重要な知見でもあり、第 2 回～第 5 回における特徴量設計の妥当性を裏付ける結果となった。

以上を通して、複勝予測においては単一レースの静的な情報よりも過去成績やレース内での相対的な位置づけを表す特徴量の設計が予測性能を大きく左右すること、また分類指標の高さと実際の回収率の高さは必ずしも一致しないことが分かった。今回使用した特徴量は、過去成績や騎手・調教師の実績などを中心としており、展開予測や血統、馬場適性などの情報は考慮していない。そのため、これらの情報を追加することで、さらなる予測性能の向上が期待できる。また、今回は複勝を対象とした二値分類モデルを構築したが、購入金額の最適化や賭け方の戦略については検討していない。今後は、より多様な特徴量の追加や購入戦略の改善を行い、予測性能だけでなく実際の回収率向上を目指す必要がある。

7 意図していた実行計画との違い

当初の実験計画では、モデルの良し悪しを Accuracy・F1・ROC-AUC・Precision・Recall といった機械学習上の分類指標のみで評価し、特徴量を追加するごとにこれらの指標が向上することを目的としていた (第 1～5 回実験)。しかし、実験 5 終了時点で分類指標を確認した際、例えば ROC-AUC が 0.75 から 0.76 に上がった、F1 が 0.49 であるといった数値だけでは、そのモデルが

複勝予測として実用上「良い」のかどうかを判断することが難しいという問題に直面した。分類指標はあくまでモデルの統計的な性能を示すものであり、実際に馬券を購入した場合にどの程度の収益が見込めるかという、本来知りたい実用的な価値とは直接結びついていなかったためである。

そこで、当初の計画にはなかった評価軸として、第6回実験より複勝回収率 (実際に馬券を購入したと仮定した場合の払戻金÷購入金額) を新たに導入した。

この変更により、分類指標では優れていたモデル (XGBoost) が、回収率では必ずしも最良ではない (場面によっては Random Forest が同等以上の回収率を示す) という、分類指標だけを見ては気づけなかった知見が得られた。結果として、以降のパラメータ調整 (第7回実験) においても、分類指標と回収率の両方を確認しながらトレードオフを判断する方針に変更している。すなわち、当初は「予測精度 (分類指標) の向上」を目的としていた実行計画が、実験を進める中で「予測精度と実用的な収益性 (回収率) を両立させる」という目的へと修正された点が、当初の計画との主な違いである。

8 まとめ

本研究では、競走馬の複勝予測を目的として、過去成績や騎手・調教師情報などを利用した機械学習モデルを構築した。基本的な競走馬情報のみを用いたモデルから始め、特徴量を段階的に追加することで、どのような情報が予測性能に影響するかを検証した。

実験の結果、過去の着順や複勝率、上りタイムなどの履歴情報を追加することで予測性能が向上し、競走馬の過去の傾向を利用することが複勝予測に有効であることが分かった。また、同一レース内で特徴量を相対化することで、出走馬間の比較情報をモデルに与えることができ、さらなる性能向上につながった。

本研究を通して、機械学習ではモデルの選択やパラメータ調整だけでなく、対象データに適した特徴量を設計することが重要であることを学んだ。また、競馬のような複雑な予測問題では、予測精度だけでなく、実際の利用目的に合わせた評価方法を考える必要があることを理解した。

参考文献

- [1] Takamotoki, “JRA Horse Racing Dataset (JRA 日本中央競馬会の競馬データ),” Kaggle, 2021. [Online]. Available: <https://www.kaggle.com/datasets/takamotoki/jra-horse-racing-dataset> [Accessed: May 26, 2026].
- [2] L. Breiman, “Random Forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [3] T. Chen and C. Guestrin, “XGBoost: A Scalable Tree Boosting System,” in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining (KDD '16)*, San Francisco, CA, USA, 2016, pp. 785–794.